Celem projektu jest implementacja prostego symulatora wirtualnego świata, który ma mieć charakter dwuwymiarowej tablicy o rozmiarach NxN (domyślnie 20x20). W świecie tym będą istniały proste formy życia o odmiennym zachowaniu. Każdy z organizmów zajmuje dokładnie jedno pole w tablicy, na każdym polu może znajdować się co najwyżej jeden organizm (w przypadku kolizji jeden z nich powinien zostać usunięty lub przesunięty).

Symulator ma mieć charakter turowy. W każdej turze wszystkie organizmy istniejące na świecie mają wykonać akcję stosowną do ich rodzaju. Część z nich będzie się poruszała (organizmy zwierzęce), część będzie nieruchoma (organizmy roślinne). W przypadku kolizji następuje walka opisana w daleszej części. Kolejność ruchów organizmów w turze zależy od ich inicjatywy. Pierwsze ruszają się zwierzęta posiadające najwyższą inicjatywę. W przypadku zwierząt

o takiej samej inicjatywie o kolejności decyduje zasada starszeństwa (pierwszy rusza się dłużej żyjący). Zwycięstwo przy spotkaniu zależy od siły organizmu, choć będą od tej zasady wyjątki (patrz: Tabela 1). Okno programu powinno zawierać pole, w którym wypisywane będą informacje o rezultatach walk, spożyciu roślin i innych zdarzeniach zachodzących w świecie.

W interfejsie aplikacji musi być przedstawione: imię, nazwisko oraz numer z dziennika.

Poniższe uwagi nie obejmują wszystkich szczegółów i są jedynie wskazówkami do realizacji projektu zgodnie z regułami programowania obiektowego.

Należy utworzyć klasę **Świat** (Swiat) będącą kontenerem organizmów. Powinna zawierać m.in. metody:

- wykonajTure()
- rysujSwiat()

pola:

- organizmy

Należy również utworzyć abstrakcyjną klasę **Organizm**.

podstawowe pola:

- siła
- inicjatywa
- położenie (x,y)
- zdrowie
- nazwa
- wiek
- zręczność
- id
- szansa na rozmnożenie
- grafika
- spawnRate

podstawowe metody:

- akcja() → określa zachowanie organizmu w trakcie tury,
- kolizja() → określa zachowanie organizmu w trakcie kontaktu/zderzenia z innym organizmem,
- Mutacja() -> mutacja dla dziecka

Klasa **Organizm** powinna być abstrakcyjna. Dziedziczyć po niej powinny dwie kolejne abstrakcyjne klasy: **Roślina** oraz **Zwierzę**.

W klasie **Zwierze** należy zaimplementować wspólne dla wszystkich/większości zwierząt zachowania, przede wszystkim:

- podstawową formę ruchu w metodzie akcja() → każde typowe zwierze w swojej turze

przesuwa się na wybrane losowo, sąsiednie pole,

- rozmnażanie w ramach metody kolizja() → przy kolizji z organizmem tego samego gatunku nie dochodzi do walki, oba zwierzęta pozostają na swoich miejscach, koło nich pojawia się trzecie zwierze, tego samego gatunku.

Zaimplementuj 5 klas zwierząt (wilk, owca , jedno zwierze wymyślone przez Ciebie, 2 zwierzęta przydzielone na podstawie Twojego numeru idziennika lub inicjałów – patrz załącznik). Rodzaje zwierząt definiuje poniższa tabela.

Tabela 1. Spis zwierząt występujących w wirtualnym świecie.

Id	zwierzę	siła	inicjatywa	specyfika metody akcja()	specyfika metody kolizja()
1	wilk	9	5	brak	brak
2	owca	4	4	brak	brak
3	lis	3	7	Dobry węch: lis nigdy nie ruszy się na pole zajmowane przez organizm silniejszy niż on	brak
4	żmija	2	3	brak	Ginie przy kolizji z silniejszym przeciwnikiem, ale zatruwa pogromce, który traci 1.5 zdrowia na rok
5	żółw	2	1	W 75% przypadków nie zmienia swojego położenia.	Odpiera ataki zwierząt o sile <5. **Napastnik musi wrócić na swoje poprzednie pole.**
6	ślimak	1	1	W 90% przypadków nie zmienia swojego położenia.	Niewrażliwy na ataki zwierząt o sile <2. Ma 60% szans iż pozostanie niezauważony przez zwierzęta o sile >4. **W obu przypadkach napastnik przesuwa się na inne niezajęte pole.**
7	antylopa	4	4	Zasięg ruchu wynosi 2 pola.	50% szans na ucieczkę przed walką. Wówczas przesuwa się na niezajęte sąsiednie pole.
8	jeż	2	3	brak	Gdy ginie, tak mocno kaleczy swojego pogromcę, że ten nie może się ruszać przez kolejne dwie tury.
9	lew	11	7	brak	Król zwierząt: żadne zwierzę o sile < 5 nie ośmieli się wejść na pole zajmowane przez lwa
10	mysz	1	6	brak	Ma większą szanse na mutacje o 200%
11	komar	1	1	+1 do inicjatywy i +1 do siły za każdego sąsiadującego komara	Jeśli zostanie pokonany, ma 50% szans na przeżycie z 3 zdrowia (wraca na poprzednie pole).
12	Leniwiec	2	1	Nigdy nie przemieszcza się dwa razy pod rząd w kolejnych turach	brak

W klasie **Roślina** zaimplementuj wspólne dla wszystkich/większości roślin zachowania, przede wszystkim:

- symulacja rozprzestrzeniania się rośliny w metodzie akcja() → z pewnym prawdopodobieństwem każda z roślin może „zasiać" nową roślinę tego samego gatunku na losowym, sąsiednim polu.

Wszystkie rośliny mają zerową inicjatywę.

Zaimplementuj 4 klasy roślin (trawa, wymyślone oraz 2 rośliny przydzielone na podstawie Twojego numeru z dziennika lub inicjałów – patrz załącznik). Rodzaje roślin definiuje poniższa tabela.

Tabela 2. Spis roślin występujących w wirtualnym świecie.

roślina	siła	specyfika metody akcja()	specyfika metody kolizja()
trawa	0	brak	brak
mlecz	0	Podejmuje trzy próby rozprzestrzeniania w jednej turze	dodaje 1 zdrowie
koka	0	brak	Zwierze, które zjadło tę roślinę w następnej kolejce ma dodatkowy ruch.
guarana	0	brak	Zwiększa siłę zwierzęcia, które zjadło tę roślinę, o 3.
wilcze jagody	0	brak	Zwierze, które zjadło tę roślinę, ginie.
cierń	0	Próby rozprzestrzeniania się zawsze kończą się sukcesem.	zadaje 4 obrażenia

Stwórz klasę **Świat** zawierającą dwuwymiarową tablicę wskaźników lub referencji (w zależności od stosowanego języka programowania) na obiekty klasy **Organizm**. Zaimplementuj przebieg tury, wywołując metody akcja() dla wszystkich organizmów oraz kolizja() dla organizmów na tym samym polu. Pamiętaj, że kolejność wywoływania metody **akcja() zależy od inicjatywy (lub wieku, w przypadku równych wartości inicjatyw) organizmu.**

Organizmy mają możliwość wpływania na stan świata. Dlatego istnieje konieczność przekazania metodom akcja() oraz kolizja() parametru określającego obiekt klasy **Świat**. Postaraj się, aby klasa **Świat** definiowała jako publiczne składowe tylko takie pola i metody, które są potrzebne pozostałym obiektom aplikacji do działania. Pozostałą funkcjonalność świata staraj się zawrzeć w składowych prywatnych.
Na świecie ma się generować ilość organizmów każdego gatunku określona wzorem:

n = Floor(Sqrt(wielkosc_swiata)) * spawnRate

W aplikacji ma występować narrator, który opowiada o wydarzeniach z danej tury. Można także filtrować wiadomości np. żeby wyświetlały się tylko śmierci zwierząt, lub rozmnażanie ich.

Program ma umożliwiać edycję świata:
- dodawanie nowych zwierząt/roslin w wybrane pole
- zmiana artrybutów organizmów
- usuwanie organizmów

Inne funkcjonalności:

Walka - zwierzę może wylosować podczas Move() miejsce zajęte przez inny organizm. W tym momencie następuje walka za pomocą wzoru:

```
while(health > 0 && attacker.getHealth() > 0)
{
   if(agility*8 < r.Next(100))
   {
      this.addHealth(-attacker.getStrenght());
   }
   if (attacker.getAgility() * 10 < r.Next(100))
   {
      attacker.addHealth(-strenght);
   }
}
```

Jeśli zdrowie przeciwnika spadnie do zera lub mniej to atakujący wygrywa i przechodzi na to pole. Dodatkowo, jeśli atakujący jest mięsożerny to dostaje +5 zdrowia. Jeśli atakujący przegra to obraniający zostaje w miejscu oraz dostaje +5 zdrowia jeśli jest mięsożerny.

Śmierć - organizm żyje określoną liczbę tur. Każda tura zwiększa jego wiek o 1. Jeśli wiek przekroczy zdrowie + random(-5;5) to organizm umiera. Jeśli wiek przekroczy 100 to organizm umiera.

Dorastanie - jeśli zwierzę przekroczy 15 lat + random(-2;3) to staje się ono dorosłe i może się rozmnażać.

Prędkość - zwierzęta posiadają pole prędkość, które określa liczbę ruchów w jednej turze.

Kolejność akcji - każdy organizm wykonuje metode Akcja w swojej turze, która wygląda następująco:

1. Zwiększenie wieku
2. Sprawdzenie warunków śmierci (i dorastania dla zwierząt)
3. Ruch (zwierzęta wykonują ilość ruchów zgodną z prędkością)

Ruch - zwierze najpierw sprawdza czy wokół niego nie ma zwierząt tego samego gatuku. Jeśli jest to woli on się rozmnożyć. Jest 50% szans na udane gody. Jeśli się nie uda to zwierze losuje pole swojego ruchu ze wszystkich kierunków świata (również po skosie). Następnie sprawdza czy nie ma tam innego zwierzęcia. Jesli jest inny gatunek to następuje walka. Jeśli jest roślina - zostaje ona zjedzona (dodaje zwierzeciu +2 zdrowia). Jeśli to pole to ten sam gatunek - ruch jest pomijany.

Rozmnażanie - zwierzęta tego samego gatunku mogą się z sobą rozmnażać. Zwierzęta sprawdzają podczas ruchu czy obok nich nie znajduje się zwierze takiego samego gatunku. Jeśli tak to następuje rozmnażanie. Wzór na udane rozmnażanie:

```
randomDouble * 5 <= szansa na rozmnożenie
```

Jeśli się uda to w losowym polu obok powstaje nowa instancja tego gatunku na podstawie mutacji (patrz: Mutacja).

Mutacja - rozmnażanie organizmów powoduje mutacje. Szansa na mutacje jest określana dla całego świata i może być ustawiona na początku gry. Wzór na mutacje:

```
e.health = this.health + (r.NextDouble() * 2 - 1);
if (e.health <= 0)
   health = 1;
e.multiplyChance = this.multiplyChance + (r.NextDouble() - 0.5) / 15;
if (e.multiplyChance < 0.0)
{
  e.multiplyChance = 0.0;
}
if (this.strenght > 0)
{
  e.strenght = this.strenght + (r.NextDouble() - 0.5) / 20;
   if (e.strenght < 0.0)
      e.strenght = 0.0;
}
e.speed = this.speed + (r.NextDouble() - 0.5) / 20;
if (e.speed < 1.0)
   speed = 1.0;
```

Załącznik 1. Sposób przydziału organizmów poszczególnym dzieciaczkom.

Przydział zwierząt i roślin jest zdeterminowany numerem w dzienniku oraz inicjałami autora.

Przydział zwierząt jest realizowany w następujący sposób:

$ID_1 = (X \bmod 5) + 3,$

$ID_2 = (Y \bmod 5) + 8$

gdzie:

ID_1 – id (wg tabeli 1) pierwszego ze zwierząt

ID_2 – id (wg tabeli 1) drugiego ze zwierząt

X – przedostatnia cyfra numeru w dzienniku (jeśli jednocyfrowa => wpisz 0)

Y– ostatnia cyfra numeru z dziennika

if $(ID_1 == ID_2)$ *then*

$ID_2 = [(Y+1) \bmod 5] + 8;$

end;

Przydział roślin jest uzależniony do inicjałów pochodzących od imienia i nazwiska autora wg następującej tabeli:

Tabela 3. Zasada przydziału roślin na podstawie inicjałów imienia i nazwiska autora.

Litery od A do D	mlecz
Litery od E do H	koka
Litery od I do M	guarana
Litery od N do P	wilcze jagody
Litery od R do Z	cierń

UWAGA!

Osoby posiadające inicjały składające się z takich samych liter powinny zaimplementować roślinę która wynika z ich inicjałów oraz następną w kolejności (lub poprzednią, jeżeli według algorytmu przypada im ostatnia roślina w tabeli).

Załącznik 2. Szczegółowy zakres wymagań technicznych w projekcie

Są to warunki konieczne do spełnienia w celu uzyskania konkretnej oceny - tj. brak któregoś z elementów wymaganych na 6pkt. spowoduje uzyskanie oceny niższej niż 6pkt. Podczas oddawania student powinien również potrafić wskazać i omówić w kodzie źródłowym miejsca w których występują wymienione dalej punkty i odpowiedzieć na związane z nimi pytania.

Klasy i obiekty

1. W projekcie należy użyć klas oraz wykorzystywać obiekty, nie jest dopuszczalne pisanie "luźnych" funkcji (poza funkcją main) (konieczne na >=3pkt)
2. Logiczny podział na przestrzenie nazw - każda przestrzeń nazw w oddzielnym module (pliku) (konieczne na 3pkt)

Hermetyzacja

1. Wszystkie pola klas powinny być prywatne lub chronione (protected) (konieczne na >=3pkt)
2. Wybrane klasy powinny mieć metody typu get i set dla składowych lub tylko get lub całkowity brak dostępu bezpośredniego (konieczne na >=4pkt)

Dziedziczenie

1. Przynajmniej 1 klasa bazowa po której dziedziczy bezpośrednio (w tym samym pokoleniu) kilka klas pochodnych (konieczne na >=3pkt)
2. Wielokrotne wykorzystanie kodu (kod w klasie bazowej używany przez obiekty klas pochodnych) (konieczne na >=3pkt)
3. Nadpisywanie metody klasy bazowej wraz z wywołaniem jej w implementacji klasy pochodnej (konieczne na >=4pkt)

Kompozycja

1. Klasa (kontener) zawierający zestaw obiektów innej klasy wg przykładowego schematu: (konieczne na >=3pkt)

```
class WirtualnySwiat {
[...]
  Organizm * organizmy;
  int iloscOrganizmow;
[...]
}
```

2. Dla powyższego przykładu umożliwić co najmniej dodawanie i usuwanie obiektów z tablicy. (konieczne na >=5pkt)
3. Dla powyższego przykładu umożliwić transfer obiektu pomiędzy różnymi kontenerami bez tworzenia nowego obiektu (konieczne na >=5pkt)

Polimorfizm

1. Implementacja tablicy obiektów klasy macierzystej, w której będą przechowywane obiekty klas potomnych. Wywołanie tej samej metody na każdym polu tej tabeli (konieczne na 3pkt)

Inne wymagania

1. Stan wszystkich obiektów (w tym kontenerów) powinien wczytywać i zapisywać się do pliku (konieczne na 5pkt)
2. Implementacja własnych konstruktorów kopiujących implementujących kopiowanie jeśli zwykły konstruktor kopiujący -domyślny - nie wystarcza (konieczne na >=4pkt)
3. Zaimplementować i zademonstrować własne wyjątki (konieczne na 5pkt) (w szczególności użycie try, catch, throw)
4. Zakaz używania "latających" stringów (np.: if(nazwa == "wilk"))